Introduction to Artificial Intelligence

Lecture 9: Language models II

October 2, 2025

Quiz questions

1. Which of these terms best describes the type of AI used in today's email spam filters, speech recognition, and other specific applications?
    - Artificial narrow intelligence (ANI)
    - Artificial general intelligence (AGI)

2. What do you call the commonly used AI technology for learning input (A) to output (B) Mappings?
    - Artificial general intelligence
    - Reinforcement learning
    - Supervised learning
    - Unsupervised learning

Quiz questions

3. The only way to acquire data for a supervised learning algorithm is to manually label it. I.e., given the input A, to ask a human to provide B
   - True
   - False

4. You run a company that manufactures scooters. Which of the following are examples of unstructured data
   - Audio files of the engine sound of your scooters
   - Pictures of your scooters
   - The maximum speed of each of your scooters
   - The number of scooters sold per week over the past week

What is data?

- Example of a table of data (dataset)

Size of house (square feet)	# of bedrooms	Price (1000$)
523	1	305
645	1	384
726	2	540
1088	2	653

A

B

- Acquiring data
  - Manual labeling
  - Passive observing
  - Download from Web

User ID	Time	Price (1000$)	Starred
1234	Aug 3	490	yes
9871	Sep 5	599	no
7623	Sep 20	384	yes
1097	Oct 2	999	no

Real-world data is messy

- Data problems
  - Incorrect labels
  - Missing values

User ID	# storage spaces	Air conditioning?	Price (1000$)	Starred
1234	1	yes	490	yes
9871	unknown	yes	599	no
7623	0	unknown	384	yes
1097	2	yes	99999	no

A B

- Multiple types of data
  - Images, audio, text: *unstructured data*
  - By contrast, structured data can be stored in a spreadsheet
  - Requires more specialized AI techniques for dealing with unstructured data

Lecture plan

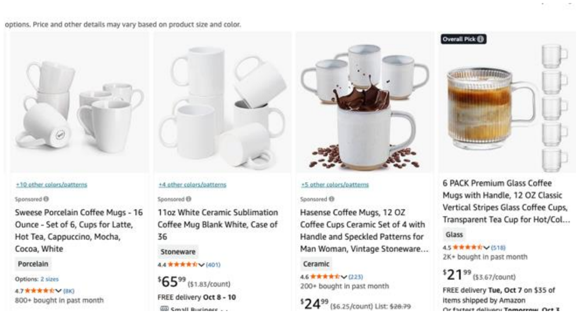
- Building AI projects (continued)
- Language models

Workflow of a data science project

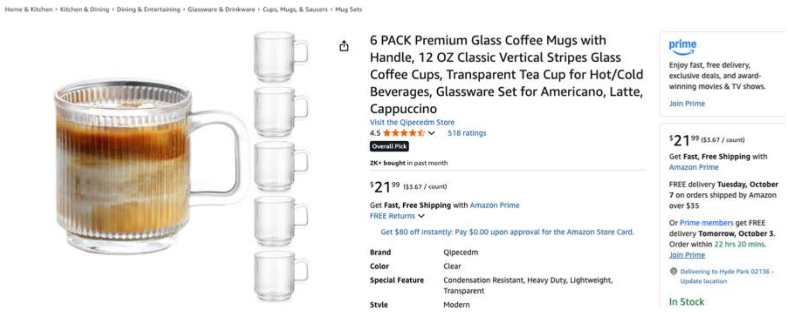
- Unless a machine learning project, the goal of a data science project is usually to get business or managerial insights by analyzing the data

- Example: optimizing a sales funnel (customer journal)

Visit website

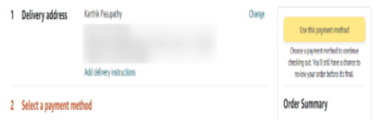


Project page



Shopping cart

Checkout



Key steps of a data science project

Optimizing a sales funnel (as many people as possible go through this process)

User ID	Country	Time	Webpage
2008	America	00:01:30 Feb 4	Amazon.com
4901	Mexico	10:48:01 Apr 20	Mug.com
9981	Brazil	20:08:19 Jun 13	Cup.com
0812	Singapore	06:35:57 Aug 30	Coffee.com

1. Collect data
2. Analyze data
   - What is affecting the performance of sales funnel? Overseas customers are not buying due to shipping costs? -> Factor in shipping cost in the price
   - Leaps in the data? More people are shopping during holidays? Time/day effects in some countries? -> Fewer adverting during those times
   - Iterate many times to get good insights
3. Suggest hypotheses/actions
   - Deploy changes; Re-analyze new data periodically

Key steps of a data science project

Optimizing a manufacturing line: coffee mugs

1. Collect data: how much clay, how long under fire, how much moisture/temperature
2. Analyze data
   - Iterate many times to get good insights: humility too low, temperature too hot -> adjust humility, temperature depending on time of day
3. Suggest hypotheses/actions
   - Deploy changes
   - Re-analyze new data periodically: keep optimizing production line to max yield

Learning how to use data

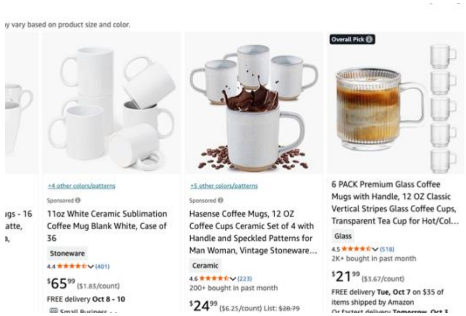
- Machine learning and data science are increasingly used in our daily lives across all kinds of industries
- Data are increasingly handed in digital records: surveys, doctor's prescription notes

Data science vs. machine learning

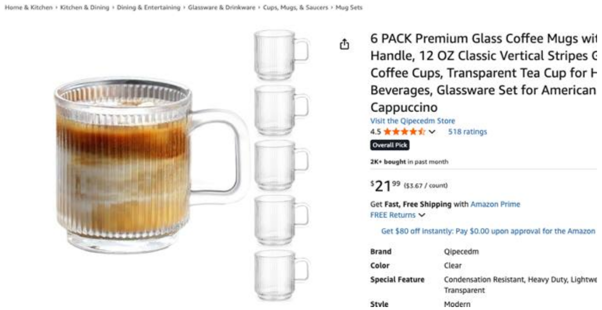
- Sales

Data science

Visit website

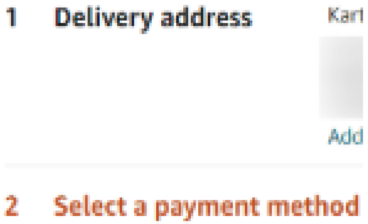


Project page



Shopping cart

Checkout



Optimizing a sales funnel

Machine learning

Name	Title	Company size	Email	Priority
Taylor	CEO	3050	tay@a..	high
Janet	Manager	230	jan@b..	medium
David	Intern	30	dave@c..	low

Automated lead role sorting

Examples (manufacturing)

Manufacturing line managers

Data science

Mix clay

Shape mug

Add glaze

Fire

Final inspection

Optimizing a manufacturing line

Machine learning

ok

crack

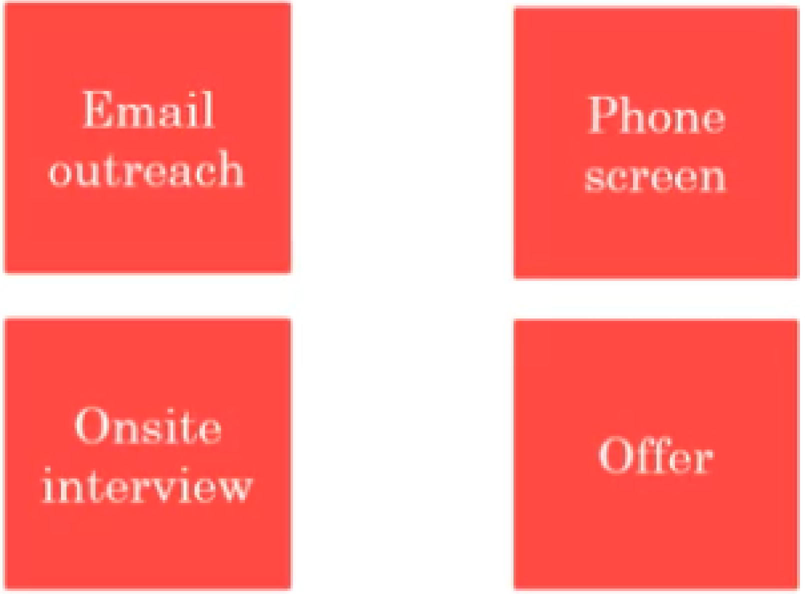
defect

Automated visual inspection

Examples (human resource)

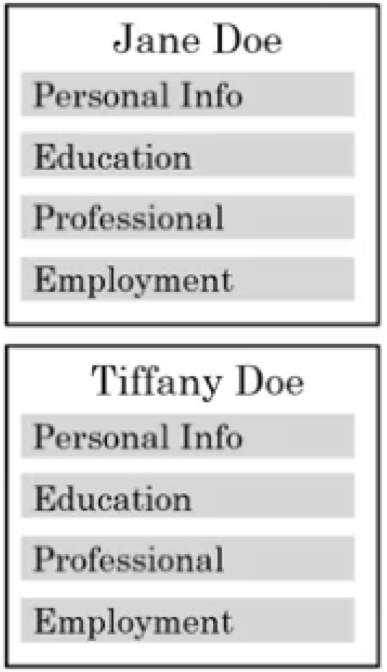
Recruiting

Data science



Optimizing recruiting

Machine learning



yes

no

Automated resume screening

Examples (marketing)

Website visits

Data science

A

B

A/B testing and online experimentation

Machine learning

Customized product recommendation

Examples (agriculture)

Agriculture

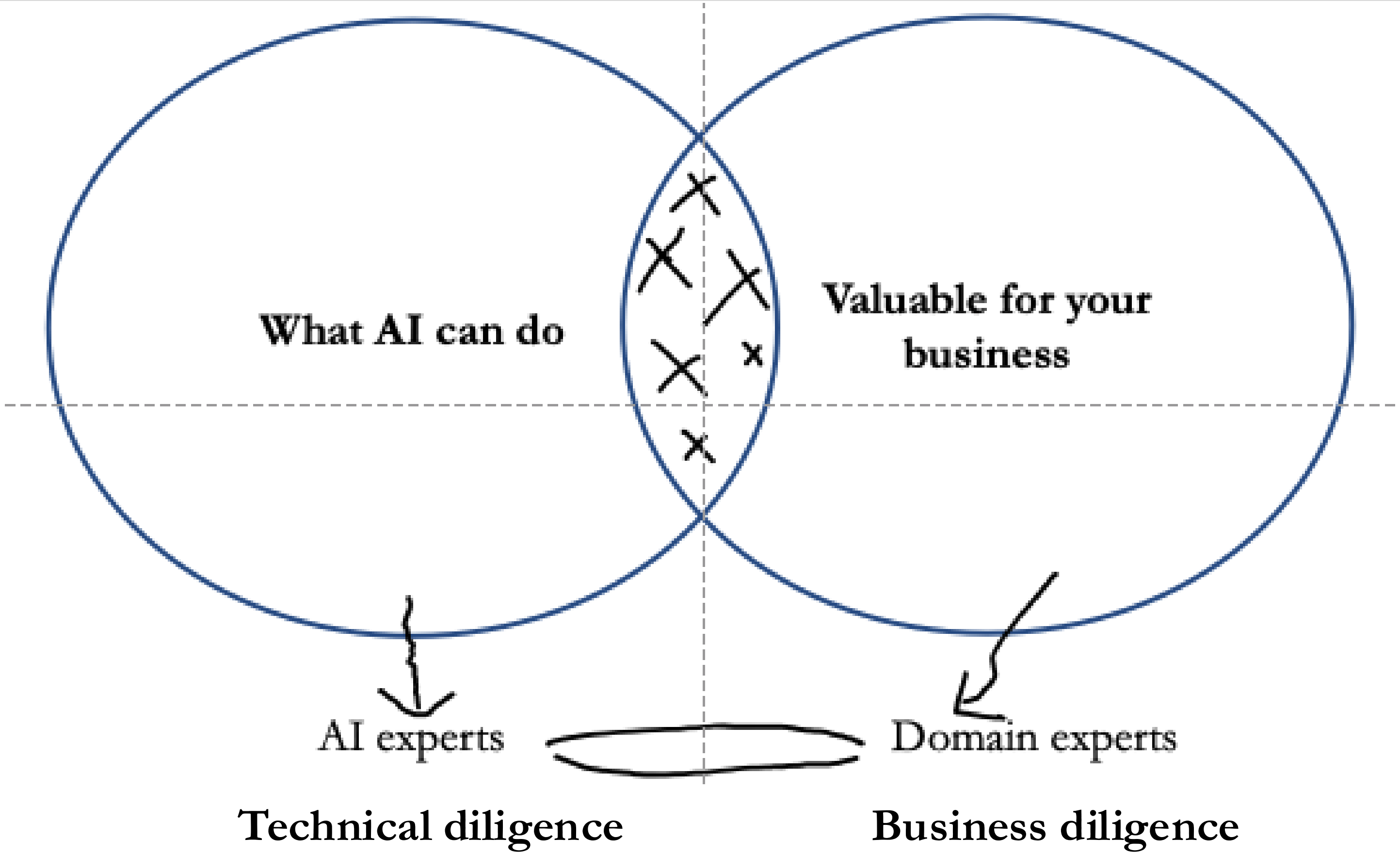
Data science

Crop analytics

Machine learning

Precision weed killing/tracking

How to choose an AI project



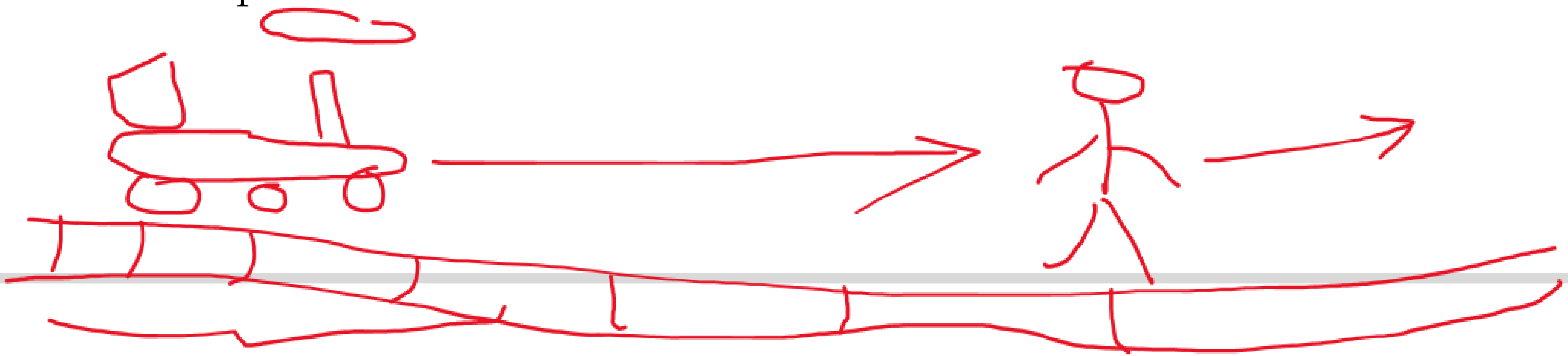
Due diligence on project

- Technical diligence
  - Can AI system meet desired performance (High accuracy)
  - How much data is needed
  - Engineering timeline

- Business diligence
  - Lower costs (automating a new task)
  - Increase revenue (driving more people to checkout a shopping cart)
  - Launch new product or business (building a new AI system)

Build vs. buy

- Machine learning projects can be in-house or outsourced
- Data science projects are more commonly in-house
  - More closely tied to business
- Some things will be industry standard – avoid building those
  - Build things that are more specific to your team
  - “Don’t sprint in front of a train”



Lecture plan

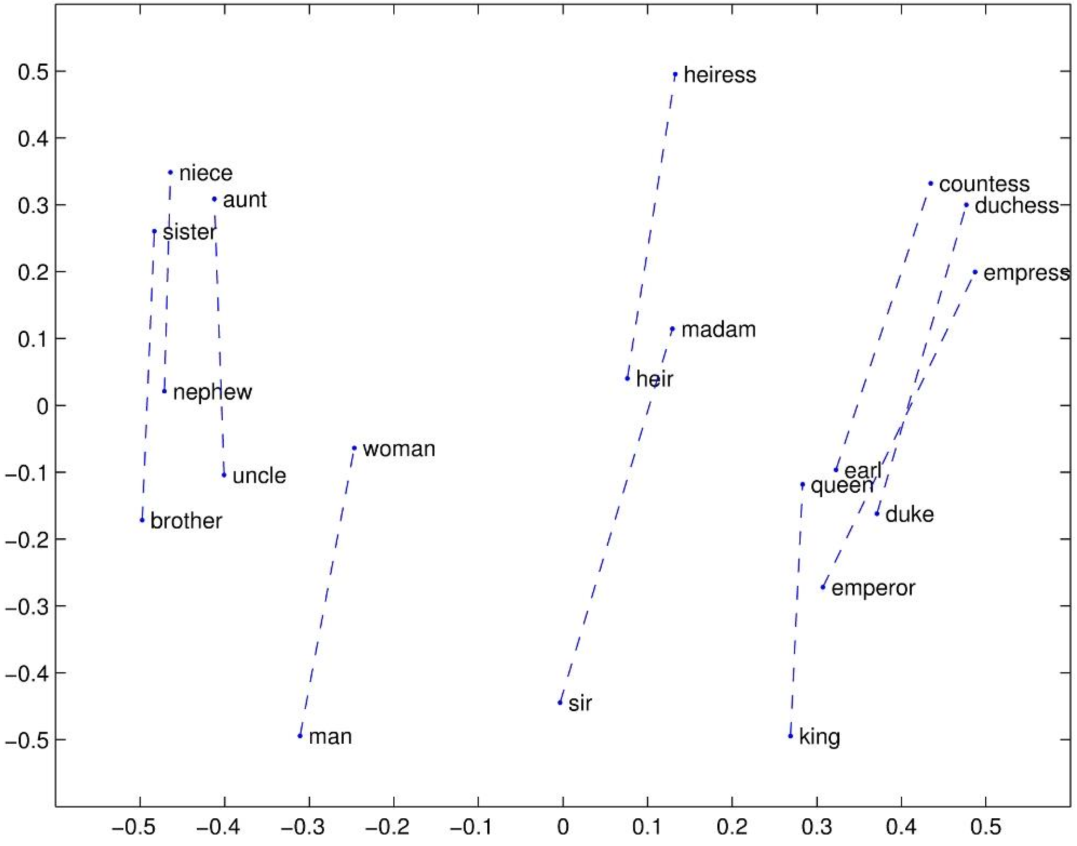
- Language models
  - What is a language model (last lecture)
  - Capabilities of a language model (last lecture)
  - **More specifics of modeling**
  - Training and adaptation

Tokenization

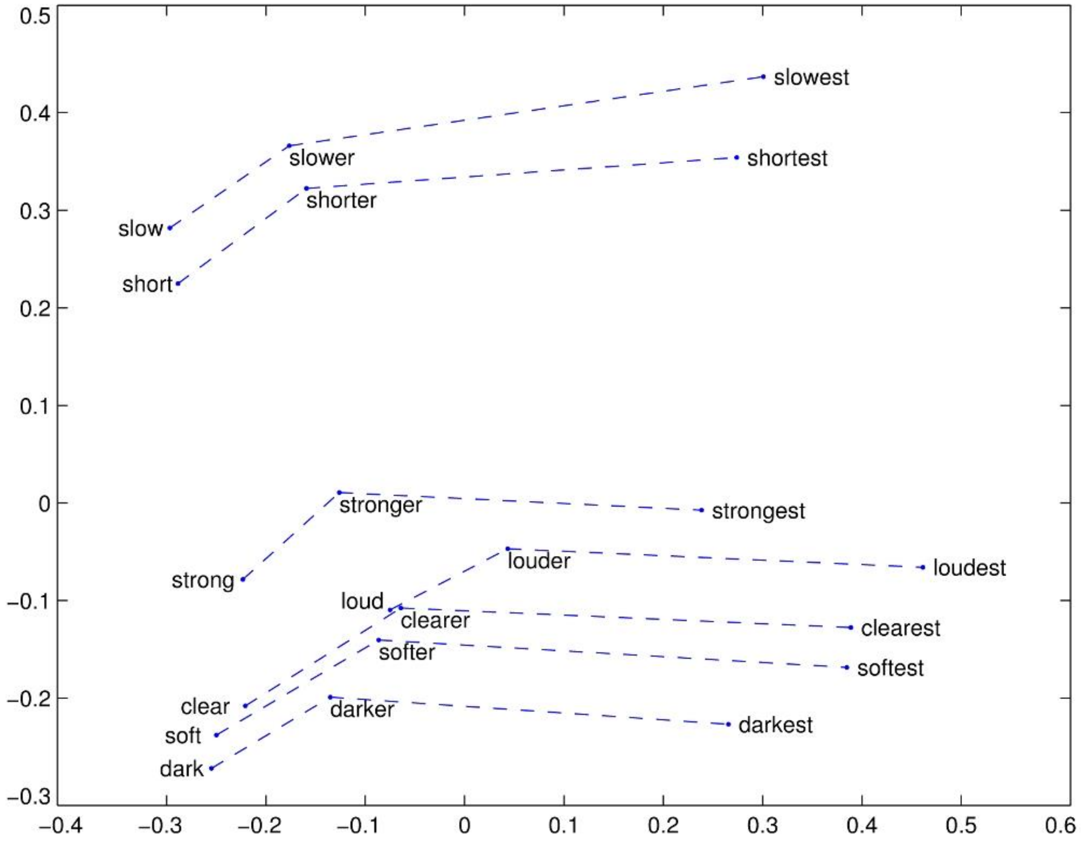
- Recall that a language model p is a probability distribution over a **sequence of tokens** where each token comes from some vocabulary, e.g., {the, mouse, ate, the, cheese}
- A **tokenizer** converts any string into a sequence of tokens
  - the mouse ate the cheese ⇒ [the, mouse, ate, the, cheese]
  - Replace each word with a word vector
- **Word embeddings**: vectors for word representation

Word embeddings can indicate linear substructures

- Male vs. female relationship



- Comparative - superlative



What makes a good tokenizer?

- Split by spaces: text.split(‘ ’)
  - However, this doesn’t work for Chinese
  - There are hyphenated words (e.g., fine-tuning) and contractions (e.g., don’t)
- **Learning the tokenizer**:
  - **Intuition**: start with each character as its own token and combine tokens that co-occur a lot
  - **Input**: a training corpus (sequence of characters)
  - **Procedure**
    - Find the pair of elements x, x' that co-occur the most number of times
    - Replace all occurrences of x, x' with a new symbol xx'
    - Repeat

Tokenizer example

- Example
  - [t, h, e, _, c, a, r], [t, h, e, _, c, a, t], [t, h, e, _, r, a, t]
  - [th, e, _, c, a, r], [th, e, _, c, a, t], [th, e, _, r, a, t] (th occurs 3x)
  - [the, _, c, a, r], [the, _, c, a, t], [the, _, r, a, t] (the occurs 3x)
  - [the, _, ca, r], [the, _, ca, t], [the, _, r, a, t] (ca occurs 2x)

Unigram model

- **Unigram model**: rather than just splitting by "frequency," a more "principled" approach is to define an objective function that captures what a good tokenization look like
  - Given a sequence $x_{1:L}$, a tokenization T is a set of
    $$p(x_{1:L}) = \prod_{(i,j)\in T} p(x_{i:j})$$
  - **Example**
    - **Input**: ababc
    - **Tokenization**: $T = \{(1,2), (3,4), (5,5)\}$
    - **Likelihood**: $p(x_{1:L}) = \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}$
  - **Algorithm**: optimize $p(x_{1:L})$ and T with alternative update (expectation-maximization)

Encoder models

- These language models produce contextual embeddings but cannot be used directly to generate text

$$x_{1:L} \Rightarrow \varphi(x_{1:L})$$

- The contextual embeddings can be used for classification tasks
  - Example: **sentiment classification**

    [[CLS], the, movie, was, great] ⇒ positive
  - Example: **natural language inference**

    [[CLS], all, animals, breathe, [SEP], cats, breathe] ⇒ entailment
  - Contextual embeddings can depend **bidirectionally** on both left/right context, however, they cannot naturally **generate** completions

Decoder models

- These are the standard **autoregressive language models**, which given a prompt $x_{1:i}$ produces both contextual embeddings and a distribution over next tokens x_{i+1}

$$x_{1:i} \Rightarrow \varphi(x_{1:i}), p(x_{i+1}|x_i)$$

  - Example: **text autocomplete**

$$\text{[the, movie, was, [CLS]]} \Rightarrow \text{great}$$

  - The embeddings can only depend on the left context, but it can naturally **generate** completions

Recurrent neural networks

- The basic form of an RNN simply computes a sequence of **hidden states** recursively
  - Process the sequence $x_1, \ldots, x_L$ left-to-right and recursively compute vectors $h_1, \ldots, h_L$
  - For $i = 1,2, \ldots, L$: compute $h_i = RNN(h_{i-1}, x_i)$
  - **RNN**: update hidden state h based on a new observation x
    - **SimpleRNN**: $\sigma(Uh + Vx + b)$
    - **LSTM** (long short-term memory) and **GRU** (gated recurrent unit)

Transformer networks

- **Attention mechanism**:
  - For a sequence $x_{1:L} = [x_1, \dots, x_L]$ and an arbitrary query y
  - A key matrix W_{key} and query matrix W_{query} to produce a score for every token
$$score_i = x_i^\top W_{key}^\top W_{query} y$$
  - Apply softmax to form a probability distribution over tokens
$$[\alpha_1, \dots, \alpha_L] = softmax(score_1, score_2, \dots, score_L)$$
  - Then the final output is a weighted combination over the values with a value matrix W_{value}
$$\sum_{i=1}^{L} \alpha_i (W_{value} x_i)$$

Self-attention and multi-head attention

- **Multi-head attention**:

$$[\text{Attention}(x_{1:L}, y), \text{Attention}(x_{1:L}, y), \ldots, \text{Attention}(x_{1:L}, y)]$$

- In a **self-attention layer**, we substitute x_i in for y as the query argument:

$$[\text{Attention}(x_{1:L}, x_1), \text{Attention}(x_{1:L}, x_2), \ldots, \text{Attention}(x_{1:L}, x_L)]$$

- **Final output**:

$$\sum_{i=1}^{h} W_O^i W_V^i X \cdot softmax\left(\frac{X^\top (W_K^i)^\top W_Q^i X}{\sqrt{d}}\right)$$

- Query W_Q, key W_K, and value W_V, output W_O

Improving training

- **Residual connection**: Instead of applying function f

$$f(x_{1:L})$$

Apply

$$x_{1:L} + f(x_{1:L})$$

- **Layer normalization**: Extract the mean and covariance of the input
- **Momentum:** An efficient, second-order optimization method
- **Adaptive gradient:** adjusts learning rate automatically during training
- **Positional embeddings:** For each position in the sequence, add an extra embedding vector for that position

Lecture plan

- Language models
  - What is a language model (last lecture)
  - Capabilities of a language model (last lecture)
  - More specifics of modeling
  - **Training and adaptation**

Training objectives (decoder models)

- Recall that an autoregressive language model defines a conditional distribution

$$p(x_i | x_{1:i-1})$$

- We first map the prefix sequence (prompt) to contextual embeddings $\varphi(x_{1:i-1})$
- Apply an embedding matrix E to obtain $E\varphi(x_{1:i-1})_{i-1}$
- Apply softmax to produce a distribution over x_i

$$p(x_i | x_{1:i-1}) = softmax(E\varphi(x_{1:i-1})_{i-1})$$

- Finally, apply maximum likelihood

$$\sum_{x_{1:L}} \sum_{i=1}^{L} -\log(p_\theta(x_i | x_{1:i-1}))$$

Encoder models

- Bidirectional transformer training objective:
  - **Masked language modeling**: Mask out a particular token, ask the model to predict the missing token
  - **Next sentence prediction**: BERT is trained on pairs of sentences concatenated. The goal of next sentence prediction is to predict whether the second sentence follows from the first or not

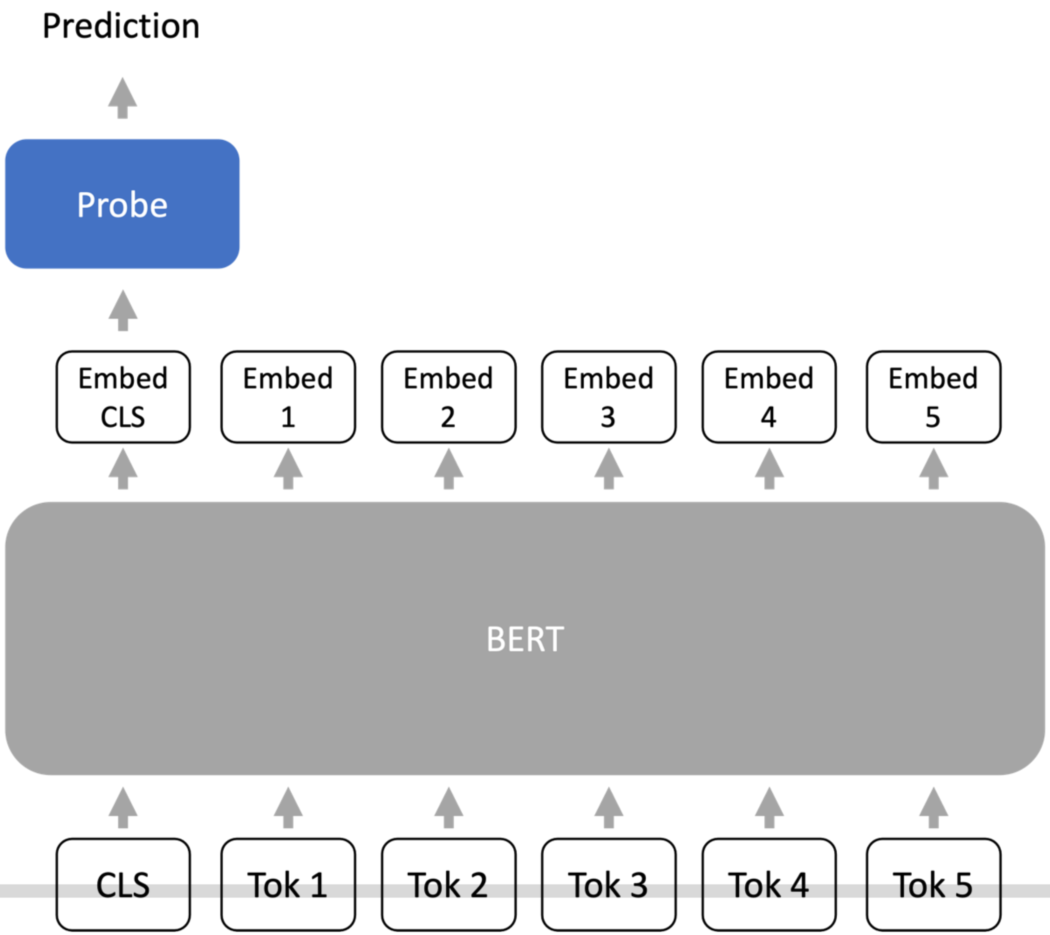
- Optimization algorithms: **Stochastic gradient**
  - Take a mini-batch of samples
  - Compute the gradient of the objective on the mini-batch, using backpropagation
  - Apply one gradient descent step with a learning rate parameter

Adapting a language model

- Language models are trained in a task-agnostic way
  - Downstream tasks can be very different from language modeling on the Pile
- Natural language inference (NLI)
  - **Premise**: I have never seen an apple that is not red
  - **Hypothesis**: I have never seen an apple
  - **Correct output**: Not entailment
- Ways downstream tasks can be different
  - **Topic shift**: the downstream task is focused on a new or specific topic
  - **Temporal shift**: the downstream task requires new knowledge that is unavailable during pre-training

Probing

- Train a probe (or prediction head) from the last layer representations of the language model to the output (e.g., class label)



Fine-tuning

- Using the entire language model parameters as the initialization or base model for optimization
- Usually, fine-tuning will produce a model that is fairly close to the base model
- Low-rank adapters and quantized adapters optimize the number of bits per performance

Transfer learning

- Transfer learning: use the information learned from one task to help learn another task
  - Example #1: building a face recognition system from open-source models plus a few hundred labeled examples
  - Example #2: fine-tuning a pre-trained language model for solving a downstream text prediction task
- Multitask learning: simultaneously train a multitask learning model on multiple objectives

LLM alignment

- Collect human-written demonstrations of desired behavior
- Perform supervised fine-tuning on the demonstrations
- On a set of instructions, sample outputs from the language model for each instruction, then gather human preferences for which sampled output is most preferred
- Fine-tune the model with a specialized objective to maximize preference reward